

Art2Sprite

Project Team

Mohammad Attique	21I-0410
Taimur Aamir	21I-0668
Rayyan M. Zia	21I-2951

Session 2024-2025

Supervised by

Ms. Marium Hida



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

Oct, 2024

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	1
1.3	Modules	2
1.3.1	Module 1	2
1.3.2	Module 2	2
1.3.3	Module 3	2
1.3.4	Module 4	3
1.4	User Classes and Characteristics	3
2	Project Requirements	5
2.1	Event Response Table	5
2.2	Functional Requirements	7
2.2.1	Data Acquisition & Preparation (Module 1)	7
2.2.2	Fine-Tuning & Data Optimization (Module 2)	7
2.2.3	Model Training & Image Optimization (Module 3)	7
2.2.4	Post-Processing & Validation Phase (Module 4)	7
2.3	Non-Functional Requirements	8
2.3.1	Reliability	8
2.3.2	Usability	8
2.3.3	Performance	8
2.3.4	Security	9
2.4	Domain Model	9
3	System Overview	11
3.1	Architectural Design	11
3.2	Design Models for Procedural Approach	12
3.2.1	Data Flow Diagram	12
3.2.2	Process Diagram	13
3.2.3	System-level Sequence Diagram	14
3.3	Data Design	15
	References	17

List of Figures

2.1	Domain Model	9
3.1	Architecture Diagram	11
3.2	Process Flow Diagram	13
3.3	System Sequence Diagram	14
3.4	Data Schema (1)	15
3.5	Data Schema (2)	15
3.6	Data Schema (3)	16

List of Tables

1.1	User Class Table	4
2.1	Event Response Table	6

Chapter 1

Introduction

This paper outlines the development and implementation plans for Art2Sprite. An AIBased Web Application for Converting Sketches to Sprite-sheets. This guide will give a detailed insight into what the goals, scope, functionality, and advantages of our final year project are. it seeks its approval to show how Art2Sprite can meet the increasing need for automation in digital sprite creation in the game industry. This document will properly focus on technical approach, impact and its alignment with market demand especially into expanding indie game sector. [2]

1.1 Problem Statement

Art2Sprite will address the problems being encountered by artists, indie game developer, and animation creator in making digital sprite sheets. The process of sprite sheet creation in the past was a complex process which took a lot of time and money, as first of all drawings are made, then colors and shading take place and then each frame is pixelated into digital sketches to be used in games. However, outsourcing of such project can be costly. This tool will help interpret sketches and create Spritesheets. This in turn enables the creator to work on their artistic version and not get fixed with technical tasks. Furthermore, it means that Art2Sprite can interconnect traditional art and video games digital creation inspiring amateurs and professionals to bring their visions to life.

1.2 Scope

Art2Sprite is an Image Diffusion and Generative Model, which will streamline the creation of sprite sheets from character sketches for game developers and artists. The core functionality of the project will be the conversion of user uploaded sketches to receive

sprites and sprites sheets ready for use in-game. Our focus will be on accurate and consistent images generation in 2D for fluid animation. The project is bound by constraints such as training the model on humanoid characters specifically, with common animation such as idle, jumping and running, ignoring more complex or unique animations for steady high quality results. This is to empower indie game developers to focus more on the development part, and artists to save resources for other parts of their games.

1.3 Modules

1.3.1 Module 1

Researching, collecting, processing and testing data even if its closely relevant to the topic. Separating the data into different categories for different stages.

1. Collecting different types of data sets on sketches and sprites.
2. Sorting them into different categories.
3. Data processing and cleaning.
4. Testing data performance.

1.3.2 Module 2

Fine tuning the image to image model for turning sketches into singular sprites. Categorize the data generated from image to image model, labelling and cleaning the data.

1. Fine tuning the model.
2. Generating sprites from sketches.
3. Categorize the data from image to image model.
4. Data cleaning.

1.3.3 Module 3

Perform Image processing on the data like resolution scaling, background removal, and test different models on that data. Fine tuning and training the model on the processed data to generate sprite sheets from sprites.

1. Image processing.
2. Testing data performance.
3. Model fine Tuning.
4. Model Training.
5. Sprite Sheet generation.

1.3.4 Module 4

Post processing the output, extensive testing and validation of the whole product. Ensuring that it is in running state.

1. Post-Processing.
2. Testing and Validation.

1.4 User Classes and Characteristics

Example: User Classes and Characteristics

User class	Description
Artists and graphic designers	This user group can create hand-drawn art, but is not necessarily familiar with sprite or coding techniques. Instead, their attention will be mainly concentrated on how aesthetically pleasing and visually comprehensible their sketches could be. They will appreciate an easy-to-use interface that eliminates technical problems associated with the generation of sprites and keeps the look and feel of the artist's rendering of the sprites.
Small Studios	These users usually create low-budget games or applications and may perform several functions: development, design, and sometimes marketing. They would appreciate a cheap and expandable proposition that facilitates quick sprite creation while still granting them enough freedom to customize their assets in terms of aesthetics.

User class	Description
Hobbyists	Casual users who do not plan to target any features but do want to make sprites for their fan games or simple apps. Although they might not exert much skill in the technology, they will still experience pleasure when creative tools are employed. For kids or hobbyists, the tool needs to provide a fun and very user-friendly work environment with no complicated setup needed, so they can quickly create sprites without drowning in the complexity of the technology.
Developers	These users will expect the tool to offer and give out sprite files which can be incorporated into game engines such as Unity and Unreal game engines. They will also expect multiple sprite formats with additional features such as animated sprites, sprite-sheets, and other variations in format and resolution.

Table 1.1: User Class Table

Chapter 2

Project Requirements

2.1 Event Response Table

Event	Response	Time Constraint
User uploads a sketch	The backend system receives the image, processes it, and begins sprite generation using machine learning models.	Must start within 1 minute
Sketch processing request timeout	The backend returns a timeout error message and prompts the user to retry. Logs the error for monitoring purposes.	Response within 3 minutes
Sprite generation success	The backend sends the completed sprite image to the user interface and triggers a success notification for the user.	Response within 3 minutes
Sprite generation failure	An error is logged, and a failure notification is sent to the user. The system suggests potential fixes (e.g., uploading a clearer sketch).	Response within 1 minute
User requests image enhancement	The backend retrieves the sprite, applies enhancement algorithms (e.g., resolution or shading improvement), and returns the updated image.	Must complete within 2 minutes
Background removal request	The backend processes the image to remove the background using the trained model, returning the modified image.	Must complete within 3 minutes
Data processing batch initiated	Backend initiates batch processing on a dataset (e.g., for model testing or dataset labeling) and logs the start time.	Must begin immediately

Event	Response	Time Constraint
Data processing batch completion	Upon completion, the backend logs the time and result, notifying the user or admin of the status.	Must complete within defined dataset size limits
Authentication request by user	The backend verifies credentials and, if authenticated, grants access to the system. Otherwise, returns an authentication error.	Must respond within 1 minutes
User requests system status update	The backend retrieves and returns current system status (e.g., uptime, last failure, etc.) to the user interface.	Response within minutes
Model update or training initiated	Backend starts model training or updates, logs the event, and updates progress to system admin.	Must start within 1 minute
Error during model training	System logs the error, retries, and notifies the admin if the problem persists.	Response within 1 minute

Table 2.1: Event Response Table

[6]

2.2 Functional Requirements

2.2.1 Data Acquisition & Preparation (Module 1)

This module focuses on gathering, processing, and evaluating data that concerns sketches and sprites and also includes putting the data in different stages.

1. The system shall allow users to upload datasets of sketches and sprites.

2.2.2 Fine-Tuning & Data Optimization (Module 2)

This module is devoted to the process of fine-tuning the model of transforming sketches into sprites as well as working with the output data for its further processing.

1. The system shall create a sprite out of an uploaded or fine-tuned sketch using image-to-image model.

2.2.3 Model Training & Image Optimization (Module 3)

In this module, advanced image processing techniques are adopted to cut out the imperfections of sprites, and finally sprite sheets are composed based on the single sprites generated in previous steps.

1. The system shall allow image processing functions such as resolution scaling, background removal, and noise reduction to enhance sprite quality.
2. The system shall automatically generate sprite sheets from individual sprites, organizing them in a format suitable for game development or animations.

2.2.4 Post-Processing & Validation Phase (Module 4)

The emphasis of this module is on post-processing the sprites and sprite sheets that have just been generated, followed by detailed testing and validation to confirm that every aspect of the system's output is redeployed.

1. The system shall ensure that the product is simply operational by providing error recovery features, error logs, and error resolution details in case there are any malfunctions.

2.3 Non-Functional Requirements

This section contains the non-functional requirements concerning the A2S system. These quality requirements have been made such a way that they are specific, quantifiable, and verifiable, and therefore the system has given performance, levels and cuts in reliability as regards various aspects.

2.3.1 Reliability

The A2S system must be able to have a high reliability metric and hence have very few failures when in operation. The reliability will be assessed in terms of Mean Time Between Failures (MTBF) and any occurrence of a failure has to be reported and documented as such.

RLB-2: It shall be possible for the system, in the event of an external system failure, to restart the application and have recovered and functioning within 30 seconds of the restart this will help in avoiding workflow interruptions for the user.

RLB-3: The system will incorporate strategies for detecting faults such that failures that will occur will be detected and recorded with a 99% performance efficiency of the rate of detection.

2.3.2 Usability

A2S will put emphasis on an interface that is friendly to the users, so that the users can easily learn and efficiently use the interaction with A2S. It will also ensure that the design will avoid most of the errors and in case there are any, proper error recovery will be in place.

USE-1: It will be possible for the users of A2S to upload a hand drawn sketch, wait for a sprite to be produced, and only perform three action steps in retrieving the generated sprite.

USE-2: Users will be able to acquire sufficient knowledge on how to navigate and use the A2S system in less than 15 minutes using interactive onboarding tutorials and training.

2.3.3 Performance

The A2S application must work optimally to give quick and active results to the users. Performance is to be evaluated through checking how long certain operations would take.

PER-1: In 95% of the cases sprite generations from uploaded sketches are completed under 2 mins when the internet used is 10 Mbps or more.

PER-2: under 1 minute, without processing time, an image that does not exceed 10mb in size is processed by the system's background removal feature.

2.3.4 Security

In the workings of the A2S system, Security will be in compliance with the industry requirements so that there is no illegal access to the data of the users.

SEC-1: All administrative users of the A2S system who changes the backend or system configuration changes shall need an admin approval.

2.4 Domain Model

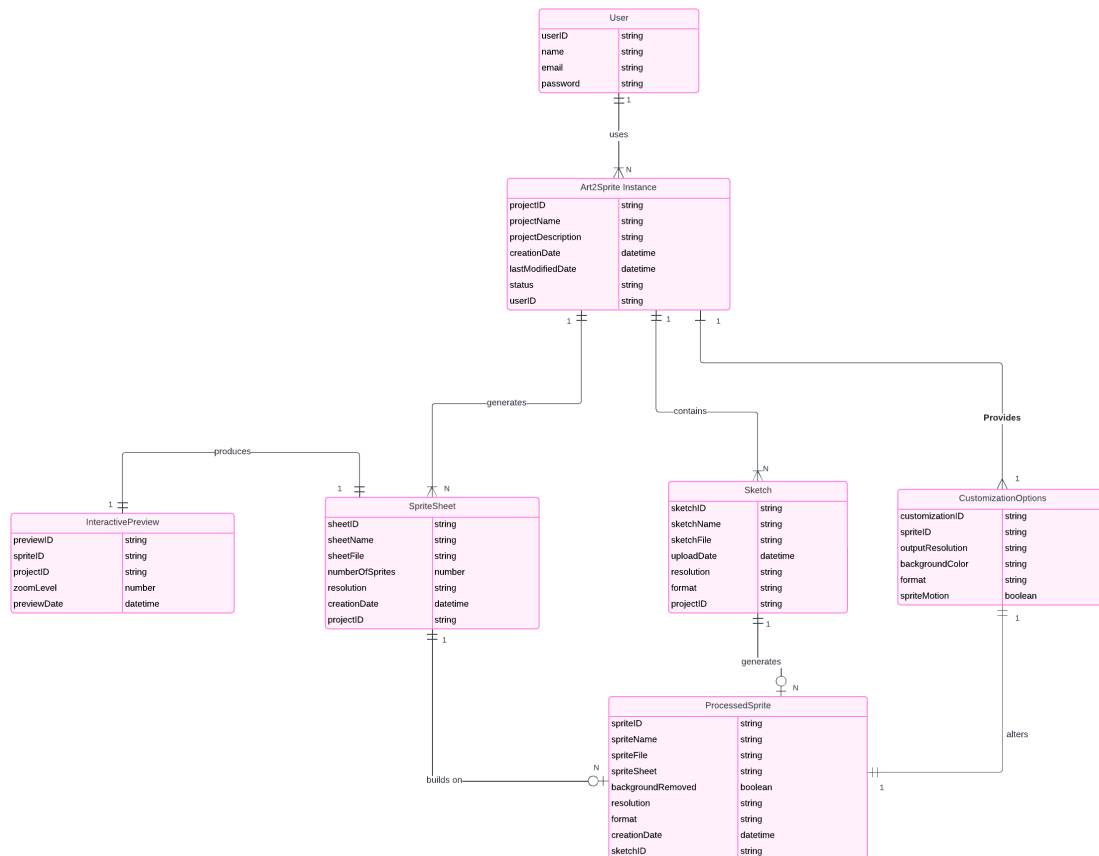


Figure 2.1: Domain Model

Our domain model represents an abstract version of the concepts and relationships between the modules and sub-modules of Art2Sprite. It demonstrates the logic, rules, and data structures inside the software, and how the 2 different AI models operate, interact and share structured data. [4]

The web app will share the information gathered from the user input, for a new instance of generating a sprite sheet. It will share the structured data - "Sketch" and "customizationOptions" - to the 1st model for "sprite generation". That will create an object of the "ProcessedSprite" class, which will then move on to the "spritesheet generation" model and back to the web, with an "interactivePreview".

This domain model will be very helpful further to build the more explanatory system designs and architecture diagrams shown in the next chapter.

System Overview

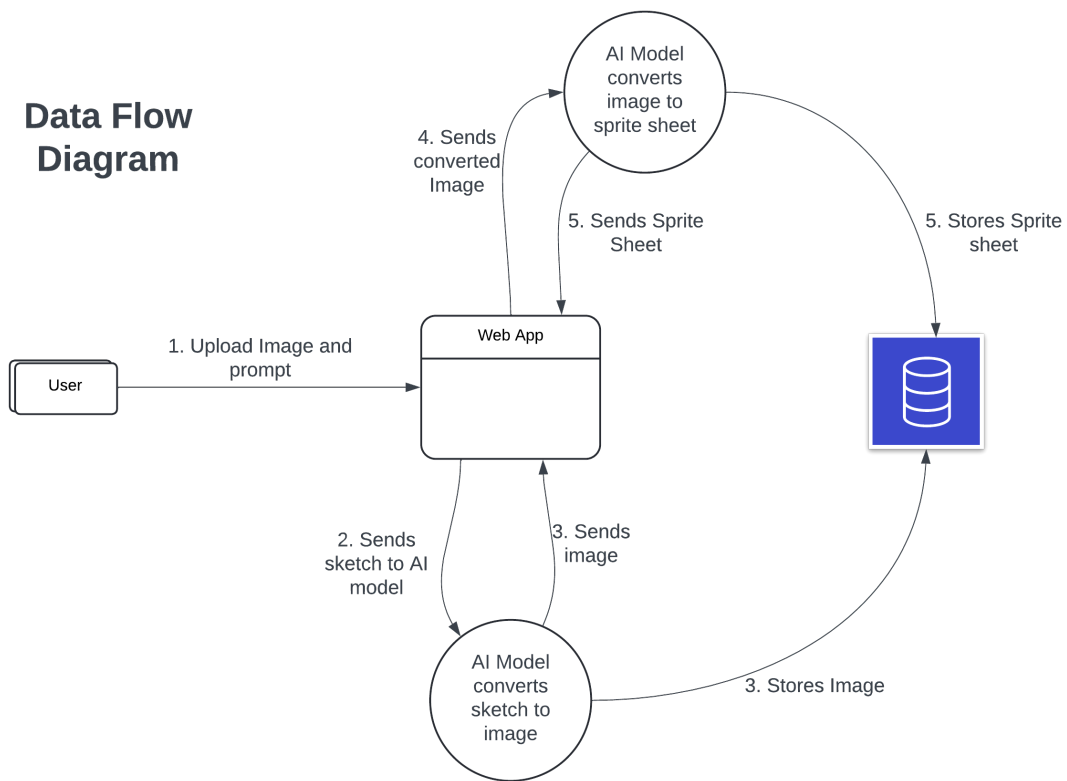
3.1 Architectural Design



The above diagram is a representation of what architecture Art2Sprite will follow. A2S will be built in the Client Server style, where 2 main modules exist: Client Side/Front End and the Server Side or Back End. The Client Side will be responsible of hosting the Web Application built with ReactJS. The Back End will be using two hosting platforms to host both servers: the AI Model on TensorFlow Serving and the Image database on Firebase. For communication between the two main modules, APIs will be used in JSON format, which is the standard HTTP file transfer protocol. [5] [3]

3.2 Design Models for Procedural Approach

3.2.1 Data Flow Diagram



The first design model for Art2Sprite is the Data flow diagram. This diagram represents how data will flow through the different high-level modules of the system, showcasing the inputs, the processing and handling location and the storage. Our system depends on inputs from users, to provide the system with a sketch and additional prompting. This is processed by the main web interface. The web interface further communicates with

two sub-modules inside the AI model for different steps of the sprite sheet generation mechanism. Both these sub-modules also handle the responsibility of storing each input and output of their system into the database for any use in future.

3.2.2 Process Diagram

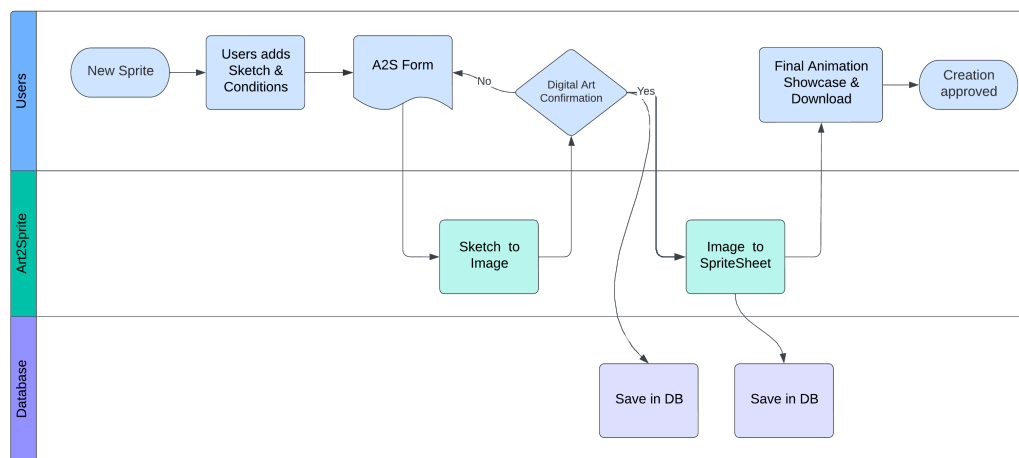


Figure 3.2: Process Flow Diagram

Next is the process flow diagram representing the sequence of operations and decision made inside the system. Each decision made or operation completed moves the process along to completion, by switching between different components where the operations are performed. Art2Sprite has three components or modules and the diagram shows the interactions between them. The user or client component is the web application responsible to start the process flow when a user requests for it. Further on, when the user is done with the A2S form, the component changes to where the next operation will take place. The AI model then begins its first task, completes it and sends it back to the first component for letting user decide if they want to keep progressing or repeat the process. The final component only gets input because nothing is being retrieved from the database during a sprite sheet generation process.

3.2.3 System-level Sequence Diagram

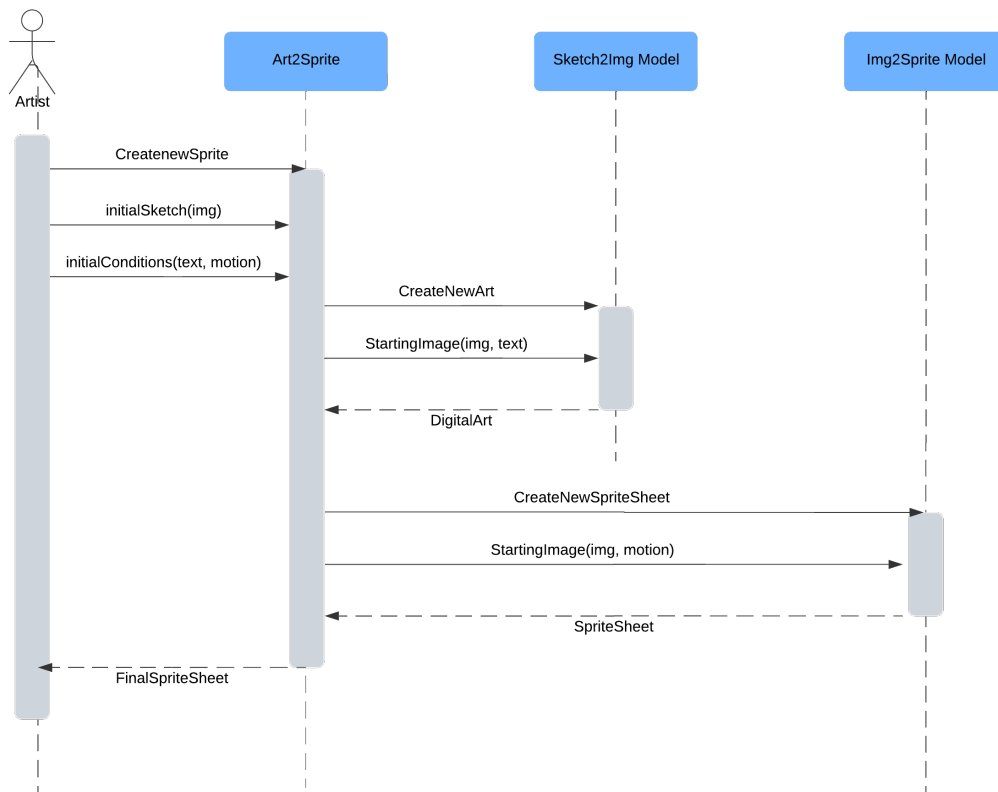


Figure 3.3: System Sequence Diagram

Going more in-depth, we come to a lower level model to show the sequential interactions between the modules by a lifeline, and using function calls, data types and return items shows up a more closer look as to how the code might look like and work like soon. This diagram is better than a standard Sequence Diagram because the system is no longer a black box, and includes the System Sequence Diagram in it, therefore called the System Level Sequence Diagram. Because Art2Sprite has sequential behavior, this diagram is a very useful design model for understanding it. Once a user creates a request on the website, and fill out the condition prompts and provide the sketch, the website will contact the first AI model to generate a digital art piece using the sketch. Once completed, the second model is asked to generate the sprite sheet using the sprite from the first model. This represents how Art2Sprite is fully optimized and cannot be optimized further, as no action can be performed in a different order, since they all depend on their previous results.

3.3 Data Design

```
{
  "New item": {
    "$schema": "http://json-schema.org/draft-07/schema#",
    "title": "Art2Sprite",
    "type": "object",
    "description": "Schema for Art2Sprite project involving user inputs, AI model processing, and output results.",
    "properties": {
      "userInput": {
        "type": "object",
        "description": "User inputs for the Art2Sprite project.",
        "properties": {
          "sketchImage": {
            "type": "string",
            "description": "URL of the sketch image uploaded by the user"
          },
          "textPrompt": {
            "type": "string",
            "description": "Text prompt describing the needed sprite art output"
          },
          "resolution": {
            "type": "string",
            "enum": [
              "Low",
              "Medium",
              "High"
            ],
            "description": "Selected resolution option for the sprite."
          },
          "frameRate": {
            "type": "integer",
            "enum": [
              12,
              24
            ],
            "description": "Selected frame rate option"
          },
          "animationType": {
            "type": "string",
            "enum": [
              "idle",
              "jump",
              "run",
              "roll",

```

Figure 3.4: Data Schema (1)

```

          "animationType": {
            "type": "string",
            "enum": [
              "idle",
              "jump",
              "run",
              "roll",
              "attack",
              "death"
            ],
            "description": "Selected animation option"
          }
        ],
        "required": [
          "sketchImage",
          "resolution",
          "frameRate",
          "animationType"
        ]
      },
      "spriteGeneration": {
        "type": "object",
        "description": "Information related to the first AI model processing.",
        "properties": {
          "generatedSprite": {
            "type": "string",
            "description": "URL of the generated sprite"
          }
        },
        "required": [
          "generatedSprite",
          "details"
        ]
      },
      "spritesheetGeneration": {
        "type": "object",
        "description": "Information related to the second AI model processing for spritesheet generation.",
        "properties": {

```

Figure 3.5: Data Schema (2)

```
    "animationType": {
      "type": "string",
      "enum": [
        "idle",
        "jump",
        "run",
        "roll",
        "attack",
        "death"
      ],
      "description": "Selected animation option"
    },
    "required": [
      "sketchImage",
      "resolution",
      "framerate",
      "animationType"
    ]
  },
  "spriteGeneration": {
    "type": "object",
    "description": "Information related to the first AI model processing.",
    "properties": {
      "generatedSprite": {
        "type": "string",
        "description": "URL of the generated sprite"
      }
    },
    "required": [
      "generatedSprite",
      "details"
    ]
  },
  "spritesheetGeneration": {
    "type": "object",
    "description": "Information related to the second AI model processing for spritesheet generation.",
    "properties": {
```

Figure 3.6: Data Schema (3)

Since Art2Sprite is a document-oriented model, we constructed a JSON schema for the data model instead of the traditional Entity Relation Diagram. Since there is only a storage for images, a traditional relational database is an excess. The JSON schema is more suitable for the structuring the data depending on the forms input and outputs from the models. [1]

Art2Sprite depends on three major communication steps, each with their own JSON schema. These are the "UserInputs", "SpriteGeneration", and "SpritesheetGeneration". Each of them is shown together above. All of the JSON elements have their specific descriptions, and we will explain the "UserInput" schema. Art2Sprite requires user input from a Web Form that receives a sketch image from the user as well as a more descriptive prompt, and the framerate, sprite resolution and animation they require. For now, we have decided this information is enough as part of dropdown options, represented by key values of each option. Each of these attributes have their own datatypes such as URI for image, string for text prompt, etcetera. All of this information is required for the AI models to work properly and produce an output which the user desires.

Creating a JSON schema is valuable for the team in the future, to ensure consistency while building APIs and validating the structure of information being shared. By enforcing these requirements, errors will be prevented during the early development process. Since using a JSON schema will standardize the data structure, all team members will understand the same schema without ambiguity or confusion and all tools and systems will also have no compatibility problems following the expected format.

Bibliography

- [1] Aditya Jadon. Understanding json modeling simplified 101. *hevodata.com*, 2022.
- [2] Jannik Lindner. Key indie game industry statistics: Teamwork, revenue, and trends. *wifitalents.com*, 2015.
- [3] Lucidchart. How to draw 5 types of architectural diagrams. *lucidchart.com/blog*.
- [4] Noberto Rodriguez. Domain modeling: What you need to know before coding. *thoughtworks.com*, 2021.
- [5] Amazon Web Services. What is architecture diagramming? *aws.amazon.org*.
- [6] Karl Wieggers and Joy Beatty. Modeling system events. *modernanalyst.com*, 2024.